SAHAYAK: Add a Moving Sign

Collect, train, test and publish the continuous model

A continuous sign includes movement. This guide shows how to add one without changing the static-sign model.

The safe order

1. Back up the movement samples and current model.
2. Record the full sign using one steady label.
3. Train a trial model.
4. Test it on fresh examples that were not used for training.
5. Make it active only if the results are acceptable.
6. Test the old and new signs.
7. Publish only the approved continuous model.

The sequence model uses 132 numbers per frame. It arranges the movement into 10 time steps: 132 x 10 = 1,320 numbers. This is the input format, not an accuracy score.

Before starting

Open VS Code PowerShell. Example project folder:
C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training

If your project is in a different folder, change the first path below. Do not reinstall packages if your current environment already works.

Step 1. Open the project and back up the old files

~~~powershell
$ProjectRoot = "C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training"
if (-not (Test-Path $ProjectRoot)) { throw "Edit ProjectRoot to your real folder." }
Set-Location $ProjectRoot
if (-not (Test-Path ".\.venv\Scripts\Activate.ps1")) { throw ".venv is missing." }
.\.venv\Scripts\Activate.ps1
python --version
python -c "import sys; print(sys.executable)"
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = ".\backups\sequence_$stamp"
New-Item -ItemType Directory -Force $backup | Out-Null
if (-not (Test-Path ".\continuous_gesture_db.db")) { throw "Continuous database missing." }
Copy-Item ".\continuous_gesture_db.db" $backup -ErrorAction Stop
if (Test-Path ".\model\sahayak_sequence.joblib") { Copy-Item ".\model\sahayak_sequence.joblib" $backup -Force }
Write-Host "Backup saved to $backup"
~~~

The continuous database is continuous_gesture_db.db. Do not replace your static CSV or a different older gesture database.

Step 2. Check old labels and sample counts

~~~powershell
@'
import sqlite3
with sqlite3.connect("continuous_gesture_db.db") as db:
rows = db.execute("SELECT label, COUNT(*) FROM sequence_samples GROUP BY label ORDER BY label").fetchall()
print("Total sequences:", sum(n for _, n in rows))
for label, count in rows:
print(f"{label}: {count}")
'@ | python -
~~~

If the database or table is missing, stop. Do not make a new database by guessing its location.

Step 3. Record the new movement

Run the collector once for each sign label.

~~~powershell
python collect_sequences.py

When asked, enter one label. Use uppercase letters and underscores, for example WHAT_IS_YOUR_NAME. Use exactly the same spelling each time. Perform the whole movement. Press C in the camera window to save one sequence. Use the on-screen quit key to finish.

Try to collect about 30 to 40 different examples for each sign if possible. Change position and speed naturally while making the same sign. Avoid saving many copies of the same movement.

Run the count check from Step 2 again. Fix spelling mistakes before training, because a typo becomes a separate label.

## Step 4. Train a trial model
This command should make a candidate file and leave the active model unchanged.
~~~powershell
python train_sequence_model.py --db
Expected candidate file: model\sahayak_sequence_candidate.joblib

Check its input size and label names:
~~~powershell
@'
import joblib
model = joblib.load(r"model\sahayak_sequence_candidate.joblib")
print("Features:", model.n_features_in_)
print("Labels:", sorted(str(x) for x in model.classes_))
'@ | python -

Expect 1,320 features and every intended continuous label. Stop if a label is missing, misspelled or unexpected.

## Step 5. Record new examples for testing
The candidate must be tested on examples it has not learned from. First save the largest current database ID:
~~~powershell
$rawId = @'
import sqlite3
with sqlite3.connect("continuous_gesture_db.db") as db:
print(db.execute("SELECT COALESCE(MAX(id), 0) FROM sequence_samples").fetchone()[0])
'@ | python -
$testStartId = [int]$rawId.Trim()
Write-Host "New test records must have IDs greater than $testStartId"

Keep this PowerShell terminal open. Run python collect_sequences.py and collect at least five NEW test examples for every sign. Do not train again before testing these examples.

Save this test script:
~~~powershell
@'
import argparse, json, sqlite3, joblib, numpy as np
from sequence_features import summarize_sequence
p = argparse.ArgumentParser()
p.add_argument("--after-id", type=int, required=True)
args = p.parse_args()
model = joblib.load("model/sahayak_sequence_candidate.joblib")
with sqlite3.connect("continuous_gesture_db.db") as db:
rows = db.execute(
"SELECT id, label, frames_json FROM sequence_samples WHERE id > ? ORDER BY id",
(args.after_id,),
).fetchall()
if not rows:
raise SystemExit("No fresh test samples found.")
X = np.asarray([summarize_sequence(json.loads(frames)) for _, _, frames in rows])
y = np.asarray([label for _, label, _ in rows])
pred = model.predict(X)
print("Fresh samples:", len(rows))
print("Accuracy:", round(float(np.mean(y == pred)) * 100, 1), "%")
for label in sorted(set(y)):
mask = y == label
print(f"{label}: {int(np.sum((pred == y) & mask))}/{int(np.sum(mask))} correct")
~~~

'@ | Set-Content ".\test_sequence_candidate.py" -Encoding utf8
python .\test_sequence_candidate.py --after-id $testStartId

```
Review the overall score and result for each sign. If a sign often fails, collect more varied samples and repeat
    training and fresh testing. Do not present one score as a promise that it will work for everyone.

## Step 6. Make the approved model active
Only do this after reviewing the fresh test results.
~~~powershell
python train_sequence_model.py --db --promote
```

The trainer should save the active model and back up the previous one. Check the active model:

~~~powershell
@'
import joblib
model = joblib.load(r"model\sahayak_sequence.joblib")
print("Features:", model.n_features_in_)
print("Labels:", sorted(str(x) for x in model.classes_))
'@ | python -

```
Run the available automatic checks:
~~~powershell
python -m unittest test_sahayak.py test_sequence.py test_sequence_db.py
```

Start the local API:

~~~powershell
python web_app.py

```
Open the local website and test the new movement and several old continuous signs. If old signs become worse,
    restore the previous model and investigate. A continuous-only change must not replace the static model.

## Step 7. Publish only the approved continuous model
Copy only model\sahayak_sequence.joblib into:
deployment\SAHAYAK-Vercel-Deployment\backend\model\sahayak_sequence.joblib

Back up the destination first. Do not copy a candidate file. Do not publish continuous_gesture_db.db, raw
    sequence samples or the static CSV. Commit only the intended model file to the branch watched by Render.
    Wait for deployment to finish.

## Step 8. Check the live backend and website
Open:
https://sahayak-backend-krh1.onrender.com/api/status

Confirm sequence_model_ready is true and sequence_sign_labels contains the new sign. Then open Live Studio and
    test both old and new movements.

## If something goes wrong
- New label missing: check the database label, active model, copied file and API status.
- Feature count is not 1,320: do not activate or publish the model.
- Fresh testing is poor: collect better examples and test again.
- Backend is not ready: wait for Render and inspect the latest logs.
- Old signs fail: restore the backup and review the changes.
- Git push fails: read the error and do not force-push.

## Final checklist
[ ] Backup made. [ ] Label spelling checked. [ ] New sequences recorded. [ ] Candidate has 1,320 features. [ ]
    Fresh test set used before retraining. [ ] Candidate activated only after review. [ ] Automated tests pass.
    [ ] Old and new signs tested locally. [ ] Only the sequence model published. [ ] Live status and Live Studio
    checked.
```
```